신용평가 스코어카드 프로젝트: 동작 원리 요약

15만 명 대출 데이터 → 점수표 → 승인 기준점 손익 분석 → 웹 시뮬레이터 → 사용자 로그 분석

0.858
스코어카드 AUC (LightGBM 0.869)

6.7% → 2.4%
승인율 80% 시 승인자 부도율

71%
사전 차단한 부도자 비율

539점
손익 최대 기준점 (전원 승인 대비 +29%)

전체 흐름

① 구간 나누기 › ② WoE 변환 › ③ 로지스틱 회귀 › **④ 점수표 scorecard.json** › ⑤ 기준점 손익 분석 › ⑥ 앱 · 이벤트 기록 › ⑦ 로그 분석 · A/B

④번 점수표 하나가 분석(노트북)과 서비스(앱)를 연결한다. 앱은 모델을 다시 학습하지 않고 점수표만 읽는다.

모델 만들기 (①~④)

1 구간 나누기 (Binning)

숫자를 그대로 쓰지 않고 구간으로 묶는다. 경계는 **의사결정나무**가 "부도자와 정상인이 가장 잘 갈리는 지점"을 찾아 정한다.

리볼빙 사용률	0~19%	19~40%	40~67%	67~87%	87~100%	100% 초과
부도율	2.0%	4.2%	8.7%	13.7%	19.1%	38.1%

구간화를 하면 세 가지 이점이 있다. **이상치가 무력화**되고(사용률 5,000,000%도 마지막 구간에 들어감), **결측과 특수코드를 따로** 둘 수 있고(소득 미기재, 연체 96·98), **비선형 관계**도 표현된다.

2 WoE 변환과 IV

WoE = ln(그 구간의 정상 고객 비중 ÷ 그 구간의 부도 고객 비중) → 양수면 안전, 음수면 위험

IV는 변수의 예측력이다. 구간들이 부도율을 얼마나 확실히 갈라놓는지를 합친 값이다. 리볼빙 사용률이 1.13으로 가장 강했고, 연체 이력 3종이 0.55~0.83, 나이가 0.25였다.

3 로지스틱 회귀 + 부호 검사

$\log(\text{부도 odds}) = b_0 + b_1 \cdot \text{WoE(리볼빙)} + b_2 \cdot \text{WoE(연체)} + \cdots$

WoE가 클수록 안전하므로 계수는 모두 **음수**여야 맞다. '보유 대출·카드 수'는 다른 변수와 정보가 겹쳐 부호가 뒤집혔다. 설명할 수 없는 점수표가 되므로 제거하고 다시 학습했다. 최종 변수는 9개다.

4 점수표로 변환 (스케일링)

log odds를 사람이 읽는 점수로 바꾼다. 기준은 두 가지다. **600점 = 정상 대 부도 50:1**, 그리고 **20점 오를 때마다 그 비율이 2배**(PDO = 20). 이렇게 하면 구간마다 정수 점수가 붙고, **총점 = 해당 구간 점수의 합**이 된다.

> 예: 리볼빙 사용률 95%는 41점 구간이다. 19% 이하 구간은 85점이다. 그래서 앱이 "사용률을 19% 이하로 낮추면 **+44점**"이라고 바로 알려줄 수 있다. 이것이 **설명 가능한 모델**의 실제 의미다.

비교 모델: LightGBM

나무 수백 개를 순서대로 쌓아, 앞의 나무가 틀린 부분을 다음 나무가 보완한다. 구간과 변수 조합을 스스로 찾아 성능이 조금 높다. 그러나 "왜 이 점수인가"를 점수표처럼 보여줄 수 없다.

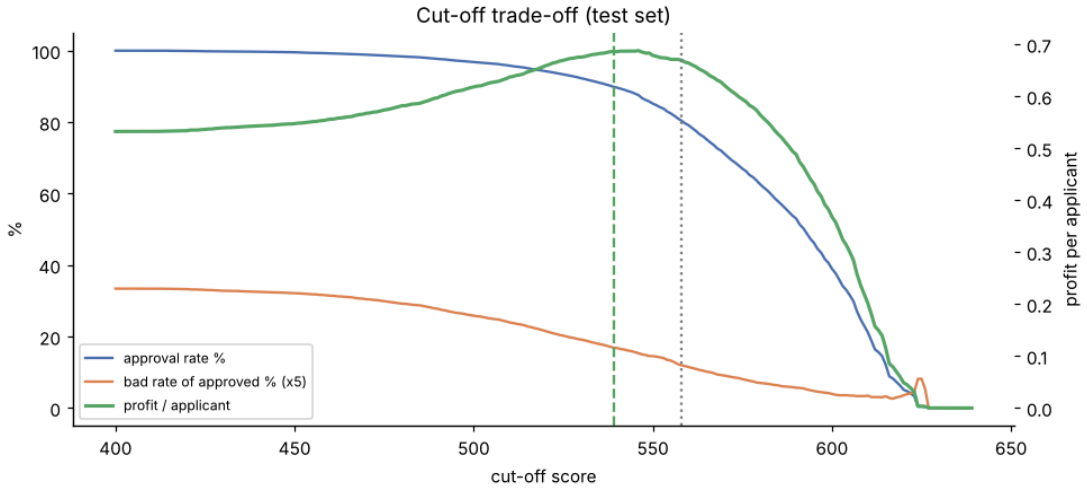
	AUC	KS	승인율 80% 시 승인된 부도자	1인당 손익
스코어카드 (운영 권고)	0.858	0.560	577명	0.670
LightGBM (챌린저)	0.869	0.583	537명	0.675 (+0.8%)

승인 기준점(cut-off) 분석 (⑤)

대출금액 정보가 없어서, 1건당 동일 금액이라는 가정 아래 손익을 정의했다.

1인당 손익 = (승인된 정상 고객 수 × **1** − 승인된 부도 고객 수 × **6**) ÷ 전체 신청자 수 (마진 10%, 손실률 60% 가정)

기준점을 400점부터 640점까지 **1점씩 올려가며** 손익을 계산했다. 기준을 낮추면 부도가 늘고, 높이면 정상 고객까지 거절한다. 그 사이에서 손익이 가장 높은 지점을 찾는다.



정책	승인율	승인자 부도율	부도 차단율	1인당 손익
전원 승인	100%	6.7%	0%	0.532
승인율 80% (558점 ↑)	80.4%	2.4%	71.2%	0.670 (+26%)
손익 최대 (539점 ↑)	89.9%	3.4%	54.6%	**0.686 (+29%)**

왜 검증셋에서 고르고 테스트셋에서 쟀나

같은 데이터로 기준점도 정하고 성과도 재면, 시험 문제를 보고 답을 고친 뒤 같은 시험으로 채점하는 것과 같다. 그래서 기준점은 검증셋에서 정하고, 성과는 처음 보는 테스트셋에서 측정했다.

민감도 분석

"손실 6배" 가정이 틀릴 수 있어서 3배~15배로 바꿔가며 다시 계산했다. 승인율 80%가 최적이 되려면 손실이 이익의 **약 10배**여야 했다. 결론: **승인율 목표는 감이 아니라 손실 구조에서 나와야 한다.**

앱과 로그 분석 (⑥~⑦)

앱 동작

점수 계산: 입력값이 어느 구간인지 찾아 점수를 더한다. 학습이 없어서 가볍다.

사용자 구분: 첫 접속 때 URL에 익명 ID(`?uid=…`)를 붙인다. 같은 링크로 다시 오면 재방문으로 집계한다.

A/B 배정: ID를 해시로 바꿔 짝수면 A(점수 중심), 홀수면 B(개선 팁 먼저)로 나눈다. 무작위이면서 같은 사람은 항상 같은 그룹이다.

이벤트 기록: 방문, 계산, 결과 조회, 피드백, 공유를 DB에 한 줄씩 쌓는다. 입력 숫자 원본은 저장하지 않는다.

로그 분석 (SQL)

퍼널: 방문 → 계산 → 결과 → 피드백 → 공유. 어느 단계에서 가장 많이 빠지는지 찾는다.

A/B 검정: 두 그룹의 "도움됨" 비율 차이가 우연일 확률(p-value)을 본다. 0.05 미만이면 진짜 차이로 판단한다.

필요 표본: 기준 35%에서 +5%p 차이를 잡으려면 **군당 약 1,400명**이 필요하다. 표본이 부족하면 판정을 미루는 것도 올바른 결론이다.

리텐션 · 유입: 첫 방문일 코호트별 재방문율, `?ref=` 링크별 전환율.

면접 대비 체크 질문

1. WoE를 왜 쓰나요? 원래 숫자를 바로 넣으면 안 되나요?
   이상치·결측 처리, 비선형 관계, 구간별 점수로 설명 가능
2. 계수 부호가 반대로 나오면 어떻게 하나요?
   다중공선성 신호 → 변수 제거 후 재학습, 설명 가능성 우선
3. AUC가 더 높은 LightGBM을 왜 안 썼나요?
   성능 차이를 손익으로 환산하면 +0.8%, 거절 사유 설명 의무 → 챌린저로 모니터링
4. 승인 기준점은 어떻게 정했나요?
   손익 함수 정의 → 검증셋에서 최적점 → 테스트셋 검증 → 민감도 분석
5. A/B 테스트가 유의하지 않으면?
   필요 표본 계산 → 기간 연장 또는 효과가 큰 지표로 재설계, 결과를 부풀리지 않음

데이터: Kaggle "Give Me Some Credit" (2011), kaggle.com/c/GiveMeSomeCredit (150,000명, 2년 내 90일+ 연체). 방법론: Siddiqi, N. (2017). Intelligent Credit Scoring, 2nd ed., Wiley. 상세 출처·해시는 docs/DATA_SOURCES.md. 분할 60/20/20 층화. 손익 수치는 동일 금액 가정에 기반한 상대 비교이며, 앱은 학습용 모델로 실제 신용평가사 점수와 다르다.